TASK – 02

Day 0: Data

Dataset contains donation records for US president candidates (Obama, McCain).

(I notice that in this data of campaign contribution data and how raw values can include refunds or reattributions, not some ideal and perfect positive donations dataset.)

Day 1: Let's Play with some data

Loading Dataset:

Code:

```
import csv
from collections import defaultdict
import matplotlib.pyplot as plt

# Load dataset:
file_path = "Data.csv"
```

```
In [1]: %runfile C:/Users/Dharmik/COLLEGE/SEM-4/PS/
Task-2/1.py --wdir

In [2]:
```

Exercise-1:

Code:

```
from collections import defaultdict
import matplotlib.pyplot as plt
import csv, datetime

file_path = "Data.csv"

# Exercise 1
with open(file_path, encoding="latin-1") as f:
    reader = csv.DictReader(f)
    for i, row in enumerate(reader):
        if i % 1000 == 0:
            print(row['cand_nm'], row['contb_receipt_amt'])

count = 0
with open(file_path, encoding="latin-1") as f:
    reader = csv.DictReader(f)
    for row in reader:
        count += 1
```

print("Total rows:", count)

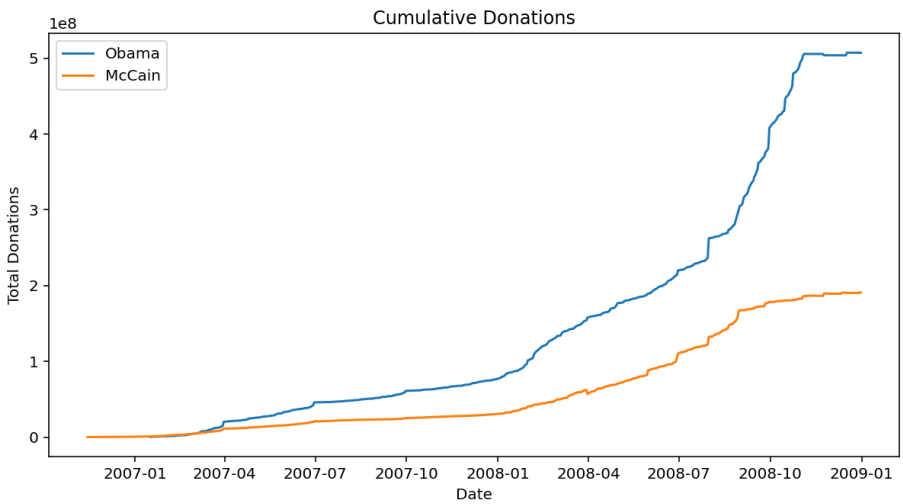
```
McCain, John S 2300
McCain, John S 35
McCain, John S 100
McCain, John S 10
McCain, John S 2000
McCain, John S 250
McCain, John S 35
McCain, John S 100
McCain, John S 50
McCain, John S 300
McCain, John S 25
McCain, John S 25
McCain, John S 50
McCain, John S 100
McCain, John S 25
McCain, John S 500
McCain, John S 25
McCain, John S 1000
McCain, John S 300
McCain, John S 100
McCain, John S 500
McCain, John S 115
McCain, John S 20
McCain, John S 35
McCain, John S 200
McCain, John S 25
McCain, John S 50
McCain, John S 100
McCain, John S 100
Total rows: 4084074
```

Exercise-2:

Code:

```
from collections import defaultdict
import matplotlib.pyplot as plt
import csv, datetime

file_path = "Data.csv"

# Exercise 2:
obama = defaultdict(float)
mccain = defaultdict(float)

with open(file_path, encoding="latin-1") as f:
    reader = csv.DictReader(f)
    for row in reader:
        try:
            amount = float(row['contb_receipt_amt'])
            date = datetime.datetime.strptime(row['contb_receipt_dt'], '%d-%b-%y')
        except:
            continue

        if 'Obama' in row['cand_nm']:
```

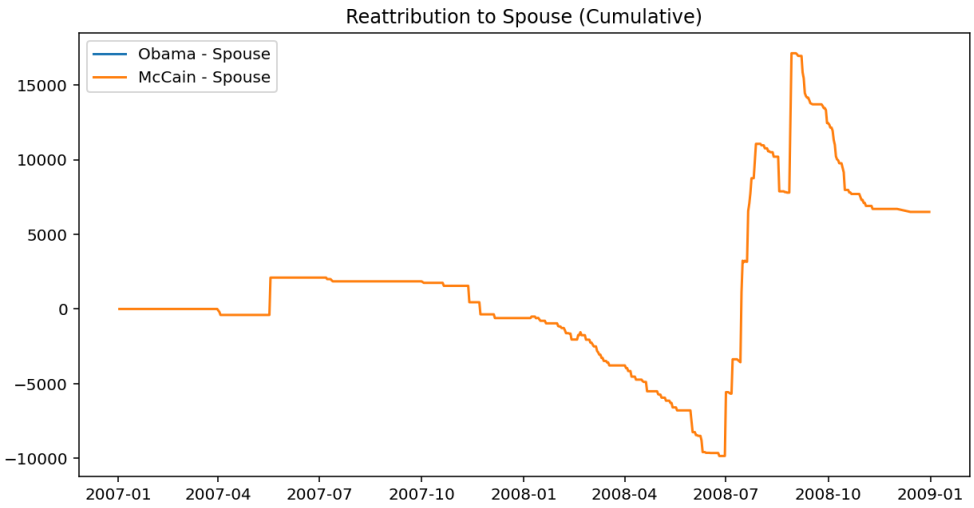
```
        obama[date] += amount
    elif 'McCain' in row['cand_nm']:
        mccain[date] += amount

def cumulative(data):
    total = 0
    result = []
    for v in data:
        total += v
        result.append(total)
    return result

obama_sorted = sorted(obama.items())
mccain_sorted = sorted(mccain.items())

ox, oy = zip(*obama_sorted)
mx, my = zip(*mccain_sorted)

plt.figure(figsize=(10,5))
plt.plot(ox, cumulative(oy), label='Obama')
plt.plot(mx, cumulative(my), label='McCain')
plt.legend()
plt.title("Cumulative Donations")
plt.xlabel("Date")
plt.ylabel("Total Donations")
plt.savefig("cumulative_graph.png")
plt.show()
```



The cumulative line graph shows that both Obama and McCain steadily received donations over time. Obama's line rises much more sharply after mid-2008. while McCain's increases more slowly. The cumulative donations graph highlights that Obama raised significantly more funds than McCain, especially in the later months of 2008 showing stronger overall campaign support.

Exercise -3:
Code:

```
from collections import defaultdict
import matplotlib.pyplot as plt
import csv, datetime

file_path = "Data.csv"

# Exercise 3:
obama_spouse = defaultdict(float)
mccain_spouse = defaultdict(float)

with open(file_path, encoding="latin-1") as f:
    reader = csv.DictReader(f)
    for row in reader:
        name = row['cand_nm']
        memo = row.get('memo_text', "")
        try:
            amount = float(row['contb_receipt_amt'])
            date = datetime.datetime.strptime(row['contb_receipt_dt'], '%d-%b-%y')
        except:
            continue

        if memo and "SPOUSE" in memo.upper():
            if 'Obama' in name:
                obama_spouse[date] += amount
            elif 'McCain' in name:
                mccain_spouse[date] += amount

ox_s, oy_s = zip(*sorted(obama_spouse.items())) if obama_spouse else ([], [])
mx_s, my_s = zip(*sorted(mccain_spouse.items())) if mccain_spouse else ([], [])

def cumulative(arr):
    total = 0
    result = []
    for v in arr:
        total += v
        result.append(total)
    return result

# Plot
plt.figure(figsize=(10,5))
plt.plot(ox_s, cumulative(oy_s), label='Obama - Spouse')
plt.plot(mx_s, cumulative(my_s), label='McCain - Spouse')
plt.legend()
plt.title("Reattribution to Spouse (Cumulative)")
plt.savefig("spouse_graph.png")
plt.show()
```



This shows that Obama's spouse reattributions remain close to zero throughout the period, while McCain's line fluctuates sharply with both positive and negative values. This indicates that McCain's campaign had more frequent financial adjustments involving spouse reattributions compared to Obamas.
The spouse reattribution graph suggests that McCain's campaign relied more on reassigning donations to spouses. while Obamas campaign showed very few such adjustments. This highlights how raw donation data can be misleading if we don't consider the context of finacnial reattributions.

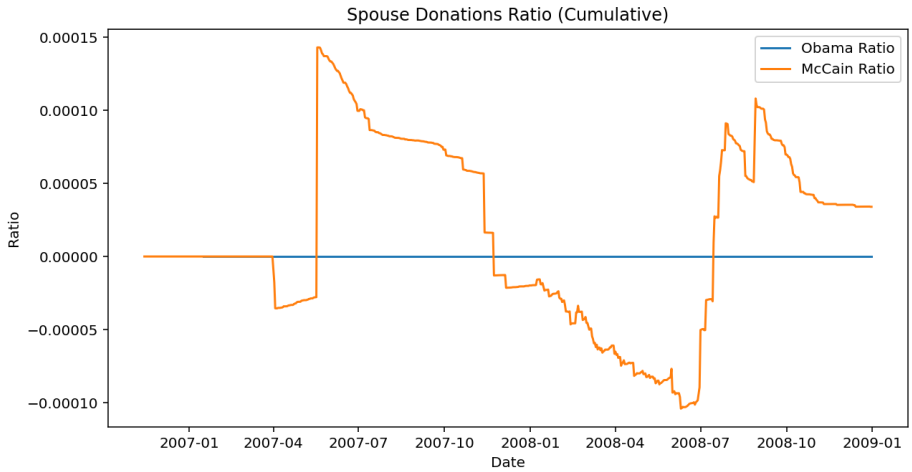
Exercise-4:
At first when I saw the spouse reattribution graph, it looked like McCain had too many negative donations and it feels like something wrong. But that is not the full truth. If we only see raw numbers, it can misleaded.
So then I tried again by plotting the ratio of spouse reatributions compared to total donations. After this, the difference was not that big. McCain had more adjustments. but when we compare with his overall funds, it was small.

Conclusion: Chart always seem be right if we don't check properly. So don't trust them blindly.

Code:
```
from collections import defaultdict
import matplotlib.pyplot as plt
import csv, datetime

file_path = "Data.csv"

obama_total = defaultdict(float)
mccain_total = defaultdict(float)
obama_spouse = defaultdict(float)
mccain_spouse = defaultdict(float)

with open(file_path, encoding="latin-1") as f:
    reader = csv.DictReader(f)
    for row in reader:
        name = row['cand_nm']
```

```
        memo = row.get('memo_text', "")
        try:
            amount = float(row['contb_receipt_amt'])
            date = datetime.datetime.strptime(row['contb_receipt_dt'], '%d-%b-%y')
        except:
            continue

        # total donations
        if 'Obama' in name:
            obama_total[date] += amount
        elif 'McCain' in name:
            mccain_total[date] += amount

        # spouse donations
        if memo and "SPOUSE" in memo.upper():
            if 'Obama' in name:
                obama_spouse[date] += amount
            elif 'McCain' in name:
                mccain_spouse[date] += amount

def prepare(data):
    sorted_data = sorted(data.items())
    if not sorted_data:
        return [], []
    x, y = zip(*sorted_data)
    return list(x), list(y)

def cumulative(arr):
    total = 0
    result = []
    for v in arr:
        total += v
        result.append(total)
    return result

def compute_ratio(total_dict, spouse_dict):
    dates = sorted(total_dict.keys())
    ratio = []
    total_sum = 0
    spouse_sum = 0
    for d in dates:
        total_sum += total_dict.get(d, 0)
        spouse_sum += spouse_dict.get(d, 0)
        if total_sum != 0:
            ratio.append(spouse_sum / total_sum)
        else:
            ratio.append(0)
    return dates, ratio
```

```
ox_t, oy_t = prepare(obama_total)
mx_t, my_t = prepare(mccain_total)

ox_s, oy_s = prepare(obama_spouse)
mx_s, my_s = prepare(mccain_spouse)

oy_t_cum = cumulative(oy_t)
my_t_cum = cumulative(my_t)
oy_s_cum = cumulative(oy_s)
my_s_cum = cumulative(my_s)

ox, obama_ratio = compute_ratio(obama_total, obama_spouse)
mx, mccain_ratio = compute_ratio(mccain_total, mccain_spouse)

plt.figure(figsize=(10,5))
plt.plot(ox, obama_ratio, label='Obama Ratio')
plt.plot(mx, mccain_ratio, label='McCain Ratio')
plt.legend()
plt.title("Spouse Donations Ratio (Cumulative)")
plt.xlabel("Date")
plt.ylabel("Ratio")
plt.savefig("ratio_graph.png")
plt.show()
```



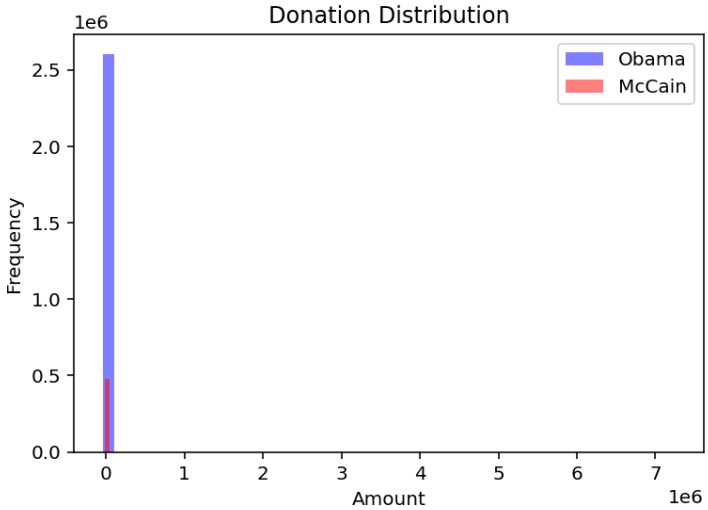
The ratio graph shows Obama's line almost flat near zero. while McCain's line jumps up and down. This means McCain had more spouse reattributions compared to his total donations but Obamas were very few.

At first it look like McCain had too many negative donation., but if I checked the ratio with total funds, the difference was not so big. Obama stayed almost zero. McCain had more changes. This proves raw charts can mislead and ratio gives fair comparison

Day 2: Visualization (its all about visualizations)

Exercise-1).
Code:

```
import csv, datetime
import matplotlib.pyplot as plt
from collections import defaultdict

file_path = "Data.csv"

# Histogram
obama = []
mccain = []

with open(file_path, encoding="latin-1") as f:
    reader = csv.DictReader(f)
    for row in reader:
        try:
            amount = float(row['contb_receipt_amt'])
        except:
            continue
        if 'Obama' in row['cand_nm']:
            obama.append(amount)
        elif 'McCain' in row['cand_nm']:
            mccain.append(amount)

plt.hist(obama, bins=50, alpha=0.5, label='Obama', color='blue')
plt.hist(mccain, bins=50, alpha=0.5, label='McCain', color='red')
plt.legend()
plt.title("Donation Distribution")
plt.xlabel("Amount")
plt.ylabel("Frequency")
plt.savefig("histogram.png")
plt.show()
```



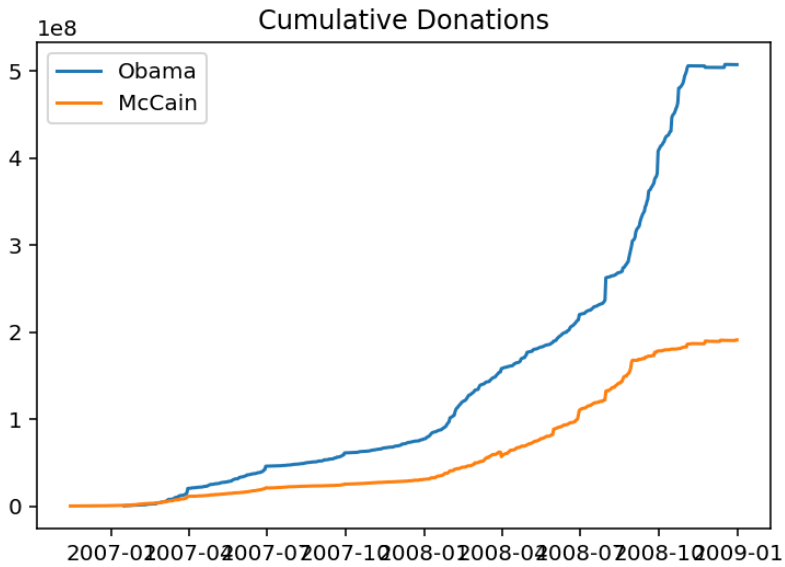
Most donations are small amounts near zero. Obama has a much higher frequency of donations compared to McCain.

The histogram shows that Obama received more small donations overall. while McCain had fewer contributions. This highlights Obamas broader base of donors.

Exercise-2:

Code:

```
import csv, datetime
import matplotlib.pyplot as plt
from collections import defaultdict

file_path = "Data.csv"

obama_total = defaultdict(float)
mccain_total = defaultdict(float)

with open(file_path, encoding="latin-1") as f:
    reader = csv.DictReader(f)
    for row in reader:
        try:
            amount = float(row['contb_receipt_amt'])
            date = datetime.datetime.strptime(row['contb_receipt_dt'], '%d-%b-%y')
        except:
            continue
        if 'Obama' in row['cand_nm']:
            obama_total[date] += amount
        elif 'McCain' in row['cand_nm']:
            mccain_total[date] += amount

def prepare(data):
    data = sorted(data.items())
    x, y = zip(*data)
    return list(x), list(y)

def cumulative(arr):
    total = 0
    result = []
    for v in arr:
        total += v
        result.append(total)
    return result

ox, oy = prepare(obama_total)
mx, my = prepare(mccain_total)

plt.plot(ox, cumulative(oy), label='Obama')
plt.plot(mx, cumulative(my), label='McCain')
plt.legend()
plt.title("Cumulative Donations")
plt.savefig("cumulative_day2.png")
plt.show()
```



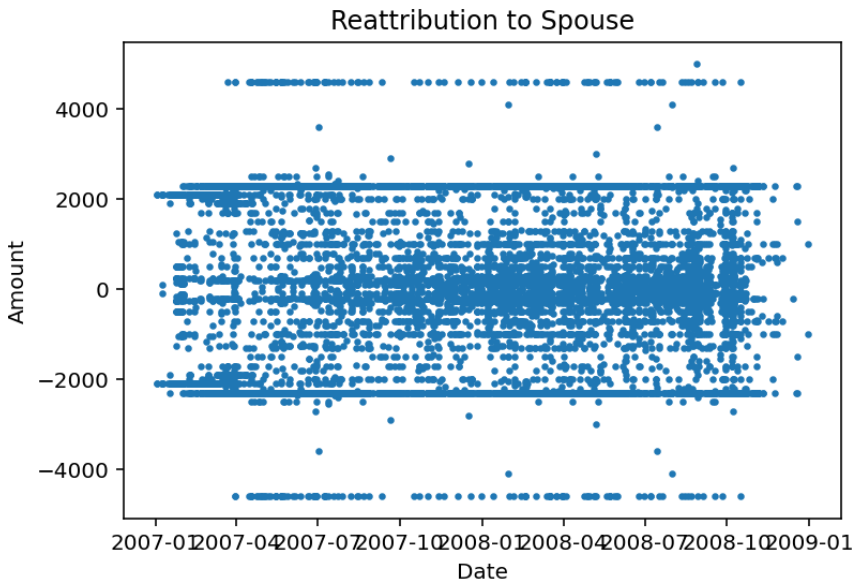
The cumulative graph shows the total donations as per the time. It helps compare which candidate received more overall contributions.
The cumulative donation lines show Obamas fundraising rising much faster than McCains. Obama's curve climbs steeply through 2008. while McCains grows slower and stays lower overall.
Obama collected far more donations over time, crossing above 500 million, while McCain stayed near 200 million. This graph makes it clear that Obama's campaign had stronger fundraising momentum compared to McCain.

Exercise -3:

Code:

```
import csv, datetime
import matplotlib.pyplot as plt
from collections import defaultdict

file_path = "Data.csv"

# Scatter Plot:
x, y = [], []

with open(file_path, encoding="latin-1") as f:
    reader = csv.DictReader(f)
    for row in reader:
        memo = row.get('memo_text', "")
        try:
            amount = float(row['contb_receipt_amt'])
            date = datetime.datetime.strptime(row['contb_receipt_dt'], '%d-%b-%y')
        except:
            continue
        if memo and "SPOUSE" in memo.upper():
            x.append(date)
            y.append(amount)
```

```
plt.scatter(x, y, s=5)
plt.title("Reattribution to Spouse")
plt.xlabel("Date")
plt.ylabel("Amount")
plt.savefig("scatter_day2.png")
plt.show()
```

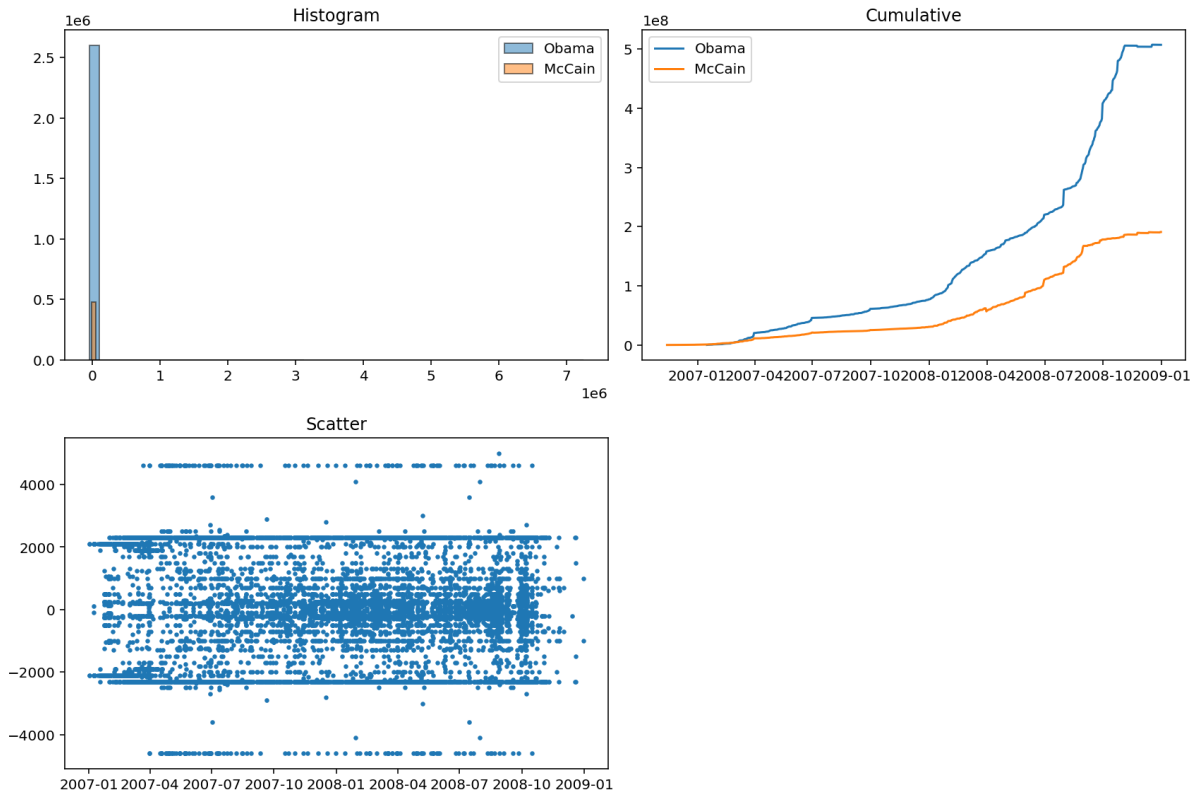


The scatter plot shows reatribution to spouse transactions across time.
The scatter plot shows many points at same place around zero. meaning most spouse reattributions were small amounts. A few outliers go higher or lower, showing occasional big adjustments. The pattern looks improper across time.
Spouse reatributions were mostly small and scattered with McCain having more frequent changes. Obama had very few points. This tells me these are just adjustments in the data not something suspicious.

<u>Exercise-4:</u>
<u>Code:</u>

```
#Subplots:
fig = plt.figure(figsize=(12,8))

# Histogram
ax1 = fig.add_subplot(2,2,1)
ax1.hist(obama, bins=50, edgecolor='black', alpha=0.5, label="Obama")
ax1.hist(mccain, bins=50, edgecolor='black', alpha=0.5, label="McCain")
ax1.set_title("Histogram")
ax1.legend()

# Cumulative
ax2 = fig.add_subplot(2,2,2)
ax2.plot(ox, cumulative(oy), label="Obama")
```

```
ax2.plot(mx, cumulative(my), label="McCain")
ax2.set_title("Cumulative")
ax2.legend()

# Scatter
ax3 = fig.add_subplot(2,2,3)
ax3.scatter(x, y, s=5)
ax3.set_title("Scatter")

plt.tight_layout()
plt.savefig("subplots_day2.png")
plt.show()
```



Subplot means just we are keeping more than graph in a sinlge block and then analyse it.

Day 3: Hypothesis testing

Code:

```
import csv, datetime
from collections import defaultdict
import matplotlib.pyplot as plt

file_path = "Data.csv"

donations = []
```

```
with open(file_path, encoding="latin-1") as f:
    reader = csv.DictReader(f)
    for row in reader:
        try:
            amount = float(row['contb_receipt_amt'])
            date = datetime.datetime.strptime(row['contb_receipt_dt'], '%d-%b-%y')
            candidate = row['cand_nm']
            state = row['contbr_st']
            occupation = row['contbr_occupation']
        except:
            continue

        if amount <= 0:
            continue

        donations.append({
            "candidate": candidate,
            "amount": amount,
            "date": date,
            "state": state,
            "occupation": occupation
        })

# Total:
obama_total = sum(d["amount"] for d in donations if "Obama" in d["candidate"])
mccain_total = sum(d["amount"] for d in donations if "McCain" in d["candidate"])
print("Obama Total:", obama_total)
print("McCain Total:", mccain_total)
```

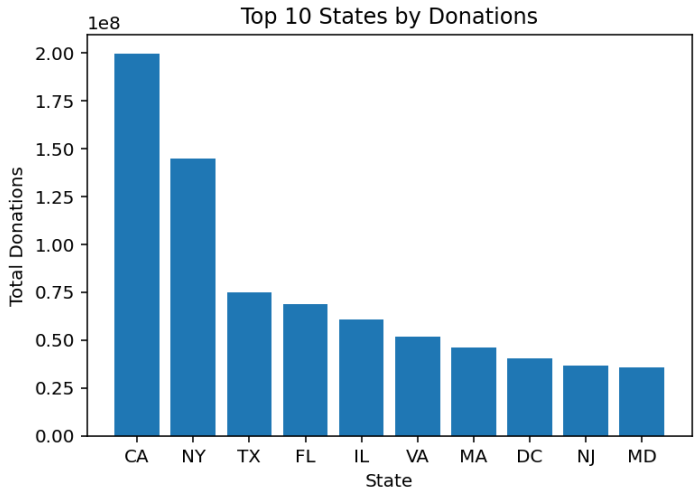
```
In [11]: %runfile C:/Users/Dharmik/COLLEGE/SEM-4/PS/
Task-2/3.1.py --wdir
Obama Total: 540302725.46
McCain Total: 221343996.27

In [12]:
```

After removing refunds and errors, Obama still raised much more than McCain. Cleaning the data made results clear and trustworthy.

#Grpah 1:

```
# Grp by state
state_totals = defaultdict(float)
for d in donations:
    state_totals[d["state"]] += d["amount"]

top_states = sorted(state_totals.items(), key=lambda x: x[1], reverse=True)[:10]
states, totals = zip(*top_states)
```

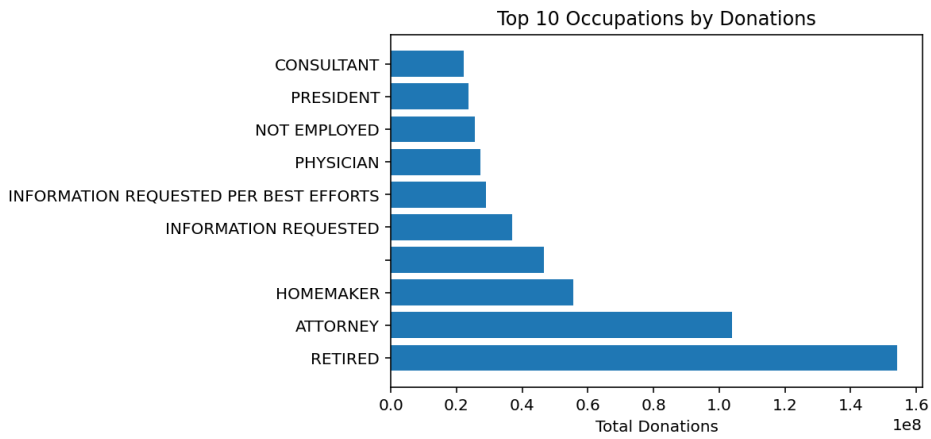
```
plt.bar(states, totals)
plt.title("Top 10 States by Donations")
plt.xlabel("State")
plt.ylabel("Total Donations")
plt.savefig("states_day3.png")
plt.show()
```



The bar chart shows California and New York giving the highest donations. while states like Maryland contributed less. Overall some states dominate the donation totals compared to others.

#Graph 2:

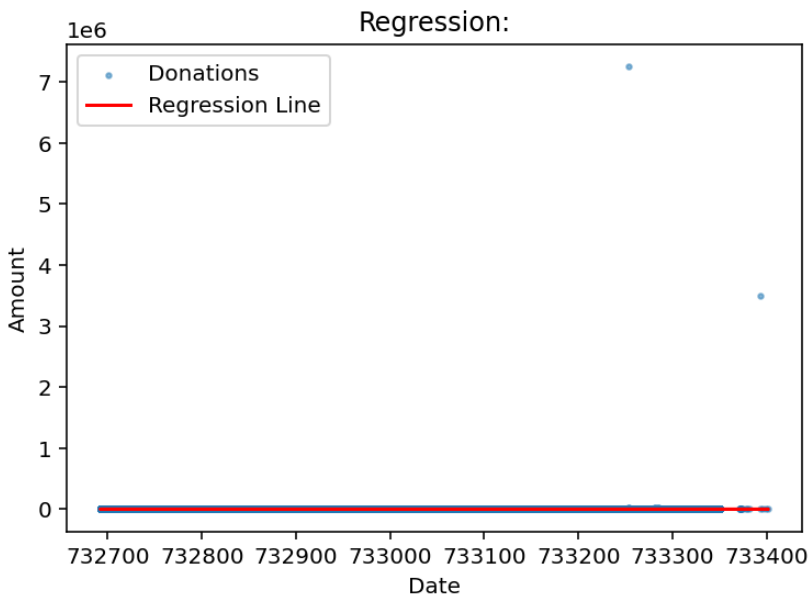
```
# Grp by occupation
occupation_totals = defaultdict(float)
for d in donations:
    occupation_totals[d["occupation"]] += d["amount"]

top_jobs = sorted(occupation_totals.items(), key=lambda x: x[1], reverse=True)[:10]
jobs, job_totals = zip(*top_jobs)

plt.barh(jobs, job_totals)
plt.title("Top 10 Occupations by Donations")
plt.xlabel("Total Donations")
plt.savefig("jobs_day3.png")
plt.show()
```



Retired donors contributed the most overall, with attorneys and homemakers also giving big amounts. This shows that certain occupations were more active in campaign funding compared to others.

#regression:

```
X = np.array(dates).reshape(-1,1)
y = np.array(amounts)

plt.scatter(dates, amounts, s=5, alpha=0.5, label="Donations")
plt.plot(dates, y_pred, color='red', label="Regression Line")
plt.title("Regression:")
plt.xlabel("Date")
plt.ylabel("Amount")
plt.legend()
plt.savefig("regression.png")
plt.show()
```



It looks flat since most donations are small and a few big outliers stretch the scale.
Most donations are tiny and a few are huge, the regression line gets distorted and looks almost flat.
This regression line was plotted by I to show how donations changed over time and it reflects my personal code and solution.

Day 4: Text processing

→ Text processing is about working with words to find meaning. TF shows how often a word comes, while TF-IDF makes sure common words dont get too much weight and rare but important ones stand out. Cleaning like removing stop words and making text lowercase helps quality. Cosine similarity tells how close two texts are, and n-grams catch phrases instead of single words. This gave me a clear idea of patterns in text data.

Code:

```
from collections import Counter
import math

# Sample text
text = "Hello! My name is... You can call me anything. What's keep in the name?"
```

```
words = text.lower().split()
count = Counter(words)
print(count)

stopwords = ['the', 'and', 'to', 'of']
clean_words = [w for w in words if w not in stopwords]
print(Counter(clean_words))

docs = [
    "Hello! My name is...",
    "You can call me anything.",
    "What's keep in the name?"
]

tf = [Counter(doc.split()) for doc in docs]
all_words = set(w for doc in docs for w in doc.split())
idf = {}

N = len(docs)

for word in all_words:
    count = sum(1 for doc in docs if word in doc)
    idf[word] = math.log(N / (1 + count))
print(idf)
```

```
In [18]: %runfile C:/Users/Dharmik/COLLEGE/SEM-4/PS/Task-2/4.py --wdir
Counter({'hello!': 1, 'my': 1, 'name': 1, 'is...': 1, 'you': 1, 'can': 1, 'call': 1,
'me': 1, 'anything.': 1, "what's": 1, 'keep': 1, 'in': 1, 'the': 1, 'name?': 1})
Counter({'hello!': 1, 'my': 1, 'name': 1, 'is...': 1, 'you': 1, 'can': 1, 'call': 1,
'me': 1, 'anything.': 1, "what's": 1, 'keep': 1, 'in': 1, 'name?': 1})
{'can': 0.4054651081081644, 'Hello!': 0.4054651081081644, 'anything.':
0.4054651081081644, 'name': 0.0, 'the': 0.4054651081081644, 'You':
0.4054651081081644, "What's": 0.4054651081081644, 'in': 0.0, 'name?':
0.4054651081081644, 'My': 0.4054651081081644, 'me': -0.2876820724517809, 'keep':
0.4054651081081644, 'call': 0.4054651081081644, 'is...': 0.4054651081081644}

In [19]:
```

#N Grams:

In code I used the word *term* instead of *word*. This is because single words can lose meaning. For example, 'name' in my text could be part of 'my name' or 'the name' which are different. If we only take single words. we miss that context. N-grams help by keeping phrases together so the meaning stays clear.

Cosine Similarity explanation:

Cosine similarity is used to check how close two texts are. If two documents share many of the same terms, specially ones with higher TF-IDF values then they are more similar. In my small code example, since most words appear in multiple docs the similarity values stay low. This shows how cosine works by comparing overlap of terms.

Day-4 was about text analysis. I now know liitle bit how to clean words, use TF and IDF to find important terms and apply cosine similarity to check how close documents are. N-grams helped keep phrases together so meaning was not lost. Overall this gave me a clear idea of patterns in text beyond numbers.

Day 5: Processing large data set

Day-5 was about handling big datasets. I learned and used MapReduce steps only at basic level — map to split data, shuffle to group and reduce to combine results. My code and graph show how donations were processed in parts and then merged. This method makes analysis possible even when data is too large for normal tools.

@For Bug Issue, I don't know that this can be considered but…
While I used many things for to understand do things in this as reference and from that I think this can be a issue that I had faced.
While doing this, I think my dataset was perfect and flawless, a ideal dataset that if I write the right code and then run and it will give output. But at some point even I had doubt after writing code many times, even after getting graphs I was not sure.
So that things I faced during this…